

Below is a **practical, industry-focused master table of commonly used design patterns** used in **day-to-day software & application development**, especially in **Java / Spring / Microservices / Enterprise systems**.

This is **not academic** — it's written the way **architects, tech leads, and senior developers** actually use patterns in real projects.

❑ CREATIONAL DESIGN PATTERNS

Pattern	Purpose	When to Use (Best Practices)	When NOT to Use	Real-World / Industry Use
Singleton	Ensure only one instance of a class	• Logging • Configuration • Cache manager • Thread-safe with enum or double-check locking	• When state changes frequently • Testing is important • Causes global state	Spring Beans (default singleton), Logger
Factory Method	Create objects without exposing instantiation logic	• Object creation logic varies • Loose coupling • Follow Open-Closed Principle	• If only one object type exists	Payment gateway creation, Notification sender
Abstract Factory	Create families of related objects	• Multiple product families • Platform-specific implementations	• Overkill for simple object creation	UI themes, DB drivers
Builder	Construct complex objects step-by-step	• Many optional fields • Immutable objects	• Simple objects with few parameters	Lombok @Builder, HTTP request builders
Prototype	Clone existing objects	• Object creation is costly • Similar objects needed	• Deep cloning is complex	Document templates, Cache cloning

□ STRUCTURAL DESIGN PATTERNS

Pattern	Purpose	When to Use (Best Practices)	When NOT to Use	Real-World / Industry Use
Adapter	Convert one interface to another	- Integrating legacy systems - Third-party APIs	You control both interfaces	Legacy API integration
Decorator	Add behavior dynamically	- Avoid subclass explosion - Flexible feature addition	Too many decorators cause complexity	Java I/O streams, Security filters
Facade	Provide simplified interface	- Reduce system complexity - Hide subsystems	When full control is required	Service layer, API gateways
Proxy	Control access to object	- Lazy loading - Security - Caching	Performance-critical hot paths	Spring AOP, Hibernate lazy loading
Composite	Treat objects and groups uniformly	Tree-like structures	Flat structures	File systems, UI components
Bridge	Separate abstraction from implementation	Multiple dimensions of variation	Simple inheritance is enough	Device-OS combinations
Flyweight	Reduce memory usage	Large number of similar objects	Objects have unique state	String pool, caching

□ BEHAVIORAL DESIGN PATTERNS

Pattern	Purpose	When to Use (Best Practices)	When NOT to Use	Real-World / Industry Use
Strategy	Select algorithm at runtime	- Multiple interchangeable behaviors - Avoid if-else chains	Only one algorithm exists	Payment strategy, Sorting
Observer	Notify multiple objects of state changes	- Event-driven systems - Loose coupling	Tight synchronous dependency needed	Kafka consumers, UI listeners

Command	Encapsulate request as object	• Undo/Redo • Queue operations	• Simple method calls	Task queues, Button actions
Chain of Responsibility	Pass request along handlers	• Multiple handlers • Decoupled request processing	• Order must be strictly controlled	Filters, Middleware
Template Method	Define algorithm skeleton	• Common workflow with variations	• Highly dynamic logic	Spring JDBC templates
State	Change behavior based on state	• State-driven behavior	• Simple conditional logic	Order lifecycle
Iterator	Traverse collections uniformly	• Hide internal structure	• Direct access is needed	Java Collections
Mediator	Centralize communication	• Reduce object dependencies	• Simple direct interactions	Chat systems
Memento	Capture object state	• Undo/Restore state	• Large object state	Versioning
Visitor	Separate operations from objects	• Stable object structure	• Frequent object changes	Report generation

❑ ARCHITECTURAL & ENTERPRISE PATTERNS (VERY COMMON)

Pattern	Purpose	Best Practices	When NOT to Use	Industry Usage
MVC	Separate UI, logic, data	• Clean separation • Thin controllers	• Microservices backend	Spring MVC
Layered Architecture	Separation of concerns	• Controller → Service → DAO	• Highly distributed systems	Enterprise apps
Microservices	Independent deployable services	• Clear domain boundaries • DevOps ready	• Small teams • Simple apps	Netflix, Amazon

CQRS	Separate read/write models	• High-scale systems	• Simple CRUD apps	Banking, Event sourcing
Event-Driven Architecture	Asynchronous communication	• Loose coupling • Scalability	• Strong consistency required	Kafka, RabbitMQ
API Gateway	Central API entry	• Security • Routing	• Small systems	Microservices
Service Locator	Central service lookup	• Legacy systems	• Modern DI frameworks	Old enterprise apps
Dependency Injection	Remove hard dependencies	• Constructor injection	• Static utilities	Spring Framework

❑ ANTI-PATTERNS (IMPORTANT FOR INTERVIEWS)

Anti-Pattern	Why It's Bad	Better Alternative
God Object	Too much responsibility	SRP + Services
Spaghetti Code	No structure	Layered architecture
Copy-Paste Programming	Maintenance nightmare	Reusable components
Premature Optimization	Wasted effort	Measure first
Over-Engineering	Complex, unreadable	YAGNI principle

❑ FINAL BEST-PRACTICE ADVICE (INDUSTRY RULES)

- ✓ Use **patterns to solve problems**, not to show knowledge
- ✓ Prefer **composition over inheritance**
- ✓ Start simple → refactor to patterns
- ✓ Overuse of patterns = **bad design**
- ✓ Clean code > fancy patterns

If you want next:

- ✓ **One single MASTER INTERVIEW TABLE**
- ✓ **Which patterns are used in Spring Boot internally**
- ✓ **Real project mapping (Pattern → Microservice component)**
- ✓ **Java code examples for each pattern**
- ✓ **When interviewers REJECT pattern usage**